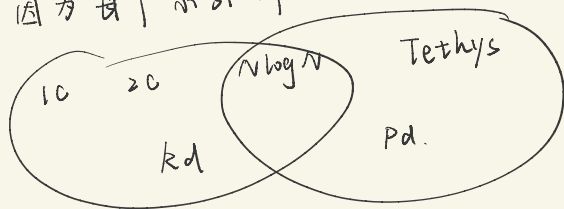


早期的论文中认为，前向安全与 I/O 效率是不可调和的
而这篇论文通过对 SDA / SDD 两个方案作了一些修改
在表现比较好的 I/O 效率的同时，还取得了前向与
后向安全。

对 SDA 的改进相对简单一些，论文发现只要将 I/O 效率
较好的静态方案应用到 SDA 中时，SDA 框架能够
产生一个具有良好 I/O 性能的动态方案

对 Rd 和 Pd 会额外引入一个 $O(\log N)$ 因子。
因为每个索引都是独立查询的



1C 方案。

Asharov 等人 [6] 提出了一个选项分配方案（我们将其称为 1C），该方案提供了最佳的局部性、最佳的空间开销和 $O(\log N \log N)$ 读取效率。给定一个大小为 N 的数据库，该方案分配一个 $m = N / (\log N \log \log N)$ bin 的数组，每个 bin 的大小为 $3 \log N \log N$ 。对于每个关键字 w ，它计算一个哈希值 $h(w)$ ，并将第 i 个文档标识符存储在 $\text{bin}((h(w) + i) \bmod m)$ 中。如果 bin 未满，则填充虚拟条目。1C 的正式描述在扩展版本中提供。

$$O(N) \quad O(1) \quad \tilde{O}(\log N)$$

2C 方案。

双选择分配 (2C)。对于大小为 N 的数据库，该方案提供了最佳局部性和空间开销，并假设关键字列表大小 $< N^{1-1/(\log \log N)^4}$ ，具有 $O((\log \log N)^4 (\log \log \log N)^2)$ 的读取效率，其中常数 $A = 1$ 。该方案分配了一个 $m = N / (\log N)^4 (\log \log \log N)^2$ 的数组，每个桶的大小为 $z \cdot (\log \log N)^4 (\log \log \log N)^2$ ，其中 $2 \leq z \leq 4$ （我们使用 $A = 1$ ）。关键字列表被填充为最接近的 2 的幂，并按降序存储。对于 w 的列表，首先将桶分成 $sb = \frac{m}{n_w}$ 超级桶；即每个超级桶由 n_w 个桶组成。然后使用两个哈希函数选择两个超级桶： $h(w) \% sb$ 和 $h_2(w) \% sb$ 。最后，负载最小的超级桶存储 w 的列表。

$$O(N) \quad O(1) \quad \tilde{O}(\log \log N)$$

ALGORITHM 3.1 (Algorithm

OneChoiceAllocation($m, (n_1, \dots, n_k)$))

- **Input:** Number of bins m , and a vector of integers $(n_1, \dots, n_k)$ representing the length of the lists $L_1, \dots, L_k$.
- **The algorithm:**
 1. Initialize m empty bins $B_0, \dots, B_{m-1}$.
 2. For each list L_i where $i = 1, \dots, k$:
 - (a) Sample $\alpha \leftarrow \{0, 1, \dots, m-1\}$.
 - (b) For $j = 0, \dots, n_i-1$:
 - i. Place the j th element of the list L_i in bin $B_{\alpha+j \bmod m}$.

ALGORITHM 3.4 (Algorithm

TwoChoiceAllocation($m, (n_1, \dots, n_k)$))

- **Input:** Number of bins m and a vector of integers $(n_1, \dots, n_k)$ representing the length of the lists $L_1, \dots, L_k$. We let $n = \sum_{i=1}^k n_i$, and assume for concreteness that m and the n_i 's are powers of two, and that $m \geq n_1 \geq n_2 \geq \dots \geq n_k$.
- **The algorithm:**
 1. Initialize m empty bins $B_0, \dots, B_{m-1}$.
 2. For each list L_i where $i = 1, \dots, k$:
 - (a) Choose $\alpha_1, \alpha_2 \leftarrow \{0, \dots, \frac{m}{n_i} - 1\}$ independently and uniformly at random. Consider the two super bins $\tilde{B}_{\alpha_1} = (B_{n_i \cdot \alpha_1 + j})_{j=0}^{n_i-1}$ and $\tilde{B}_{\alpha_2} = (B_{n_i \cdot \alpha_2 + j})_{j=0}^{n_i-1}$.
 - (b) Let $\tilde{B}_\alpha$ be the least loaded among $\tilde{B}_{\alpha_1}$ and $\tilde{B}_{\alpha_2}$. We place the list L_i in the super bin $\tilde{B}_\alpha$. That is, for every $j = 0, \dots, n_i-1$:
 - i. Place the j th element of the list L_i in the bin $B_{n_i \cdot \alpha + j}$.

$N \log N$ 方案

$N \log N$

将每个列表 $DB(w_i)$ 存储在一个表中，而不是将其拆分到 $\log N$ 个表中。

具体来说，找到一个 p_i ，使得 $2^{p_i-1} < |DB(w_i)| \leq 2^{p_i}$ ，将 $DB(w_i)$ 存在第 p_i 个表中（若不足则补齐）

为了能在查询时找到目标哈希表，方案引入一个**单独的哈希表**，其中存储每个关键词 w_i 对应的 p_i 的加密值。

- 作用：查询时，先通过这个哈希表解密得到 p_i ，就能知道要去第 p_i 个哈希表中查找该关键词的标识符列表。

这是一个空间性为 $O(N \log N)$ 、局部性为 $O(1)$ 、读取效率为 $O(1)$ 的方案。

在我们的 DSE 构造（在第3节中）中，除了加密索引外，我们还有一个加密字典。加密字典维护了一个关键字与关键字计数器的映射，即关键字 w 映射到 $|DB(w)|$ 。为了在 1C、2C、 $N \log N$ 和 sN 方案中进行搜索，需要关键字计数器（以知道需要获取多少个桶，或者搜索哪个层级）。我们将加密字典称为 EDB.DICT，将加密索引称为 EDB.IND。对于一个包含 N 个条目的索引，EDB.DICT 的大小至多为 N ，因为最多只能有 N 个不同的关键字。

SDA 的缺点就是其更新摊销
这里对 SDA 的去摊销版本 SDA 进行了改进
L-SDA
的想法就是将构建索引级别 i 的步骤拆分到
之前的 2^i 次插入中

★ 为什么要这样做？（动机）

1. 避免昂贵的集中重建

- 如果你不分摊，那么等到第 i 层需要更新时，就要一次性把 2^i 条数据合并进来。
- 这种“集中建造”会导致一次插入的代价非常大（高达 $O(2^i)$ ），破坏了算法的均摊复杂度。

2. 去摊销（de-amortization）

- 动态可搜索加密系统需要保证每次更新都高效且安全，而不是“平均下来很快”。
- 分摊后，每次插入最多只做 $O(\log N)$ 的小工作，从而保证了最坏情况更新开销也在可控范围内。
- 这就是论文里说的：L-SD 把每次更新操作拆成很多小块，逐步完成 oblMerge。

3. 保持安全性（P1-P3）

- 如果一次性集中合并，服务器能观察到“大批量数据同时被移动”，这会给攻击者更多模式信息。
- 分摊成 2^i 步小操作后，每次插入引起的服务器可见动作都差不多，降低了信息泄露风险（input-output indistinguishability）。

L-SD_d[Γ]. Update 的三条规则

- 规则一：一次新的更新由创建 NEW₀（加密索引 NEW₀.IND 和加密字典 NEW₀.DICT）来执行
- 规则二：如果 NEW_i.IND 满了，则将 NEW_i 移动到 OLDEST_i、OLDER_i 和 OLD_i 中最旧的那个空索引。在 2ⁱ 更新结束后，NEW_i.IND 变满，此时 i-1 级发生：OLD_{i-1} 移动到 OLDEST_{i-1}，OLDER_{i-1} 设为空。
- 规则三：如果 OLDEST_{i-1}.IND 和 OLDER_{i-1}.IND 都满了，那么在接下来 2ⁱ 次更新中将其中所有元素移到 NEW_i.IND 中。接下来触发规则二。

oblivious merge 协议

为了使 SD_d 满足规则三的要求，我们定义如下 oblivious merge 协议：

$$NEW_i \leftarrow \text{oblMerge}_i(K, \sigma, \text{OLDEST}_{i-1}, \text{OLDER}_{i-1})$$

如果一个静态方案 Γ.Setup 可以被转化为满足以下三个性质的 oblMerge_i，那么它就可以与 L-SD_d 的更新功能结合使用。

- **Obliviousness**（访问模式不可区分）
- **Input-Output Indistinguishability**（无法把输入条目映射到输出位置）
- **Decomposability**（合并过程可分解为多个小步，以分摊到多次更新）

oblivious merge 协议的整体想法

先将 OLDEST_{i-1}.IND 和 OLDER_{i-1}.IND 移动到一个缓冲区 BUF₁ 中，然后对 BUF₁ 中的条目重新排序，重排到符合底层静态方案 Γ（比如 1C、2C、NlogN）的 I/O 高效布局要求，然后将其移动到 NEW_i.IND 中。

缓冲区 BUF₂ 用于以 oblivious 的形式创建关键词字典。

oblivious merge 的四个阶段

1.
 - 先将 OLDEST_{i-1}.IND 和 OLDER_{i-1}.IND 复制到缓冲区 BUF₁ 中，对其按字典序做 **oblivious sort**，使相同关键字的条目相邻，dummy 条目排到末尾。然后为其分配 rank（在该关键字列表中的序号）和 n_w （该关键字列表的总长度）。
 - 对关键字列表做 padding，将其填充到最近的 2 的幂次。然后再次对其做 oblivious sort，在保持关键字字典序的同时，在不同关键字之间，按 n_w 大小降序排列。

/** Phase 1 - Preparing sorted input array */

- 1: Client parses K as (k_{rnd}, P) $\triangleright$ all encryptions and decryptions are done with key k_{rnd} , P is an array of PRF keys
- 2: Server initializes arrays **BUF₁** of size $3 \cdot N'$, and **BUF₂** of size N' to be empty at Server
- 3: Copy $\text{OLDEST}_{i-1}.\text{IND} \cup \text{OLDER}_{i-1}.\text{IND}$ to **BUF₁**; obliviously sort **BUF₁** w.r.t. lexicographic order of keywords
- 4: Perform two linear scans (one in reverse and one in correct order) on **BUF₁**, to add the **rank** and n_w values to each entry of keyword w . The entries now look like $(w, id, op, rank, n_w)$, where $0 \leq rank < n_w$.
- 5: Linearly scan **BUF₁** and pad each keyword-list with dummy elements so that its size is nearest power of 2, hide list lengths by padding total size to $2N'$
- 6: Perform oblivious sort on **BUF₁** w.r.t. lexicographic order of the keywords and descending order of the n_w values. Keep first N' elements.

2.
 - 给 BUF₁ 中的每个条目附上它在 NEW_i.IND 中的目标位置标签 (level, pos)。

1C / 2C: level = 该条目所属的 bin 号, pos 默认 0;

NlogN: level = 所在层级, pos = 在该层数组里的具体位置。

- 对于 BUF_2 , 遍历 BUF_1 中的每个条目 $(w, id, op, rank, n_w)$, 若 $rank = 0$ (即该关键字的第一条记录), 往 BUF_2 里加一条真实记录 (w, n_w, p) ; 否则, 往 BUF_2 里加一条 dummy $(\perp, \perp, \perp)$ 。

p 是一个 λ 比特长度的随机密钥。

- 对 BUF_1 中的每个条目做加密

```

/** Phase 2—Prepare index elements and prepare keyword counters */
7: for each  $j = 1 \dots |BUF_1|$  do
8:   Client decrypts  $(w, id, op, rank, n_w) \leftarrow \text{RND.Dec}(k_{rnd}, BUF_1[j])$ 
9:   Client generates  $p \leftarrow \{0, 1\}^\lambda$ 
10:  if  $rank = 0$  then Client appends  $(\text{RND.Enc}(k_{rnd}, (w, n_w, p)))$  to  $BUF_2$ 
11:  else Client appends  $(\text{RND.Enc}(k_{rnd}, (\perp, \perp, \perp)))$  to  $BUF_2$ 
12:  Client generates  $((level, pos), \sigma) \leftarrow \text{Map}(P[i][3], w, rank, n_w, \sigma)$  ▷  $pos = 0$  for 1C and 2C
13:  Client writes  $(\text{RND.Enc}(k_{rnd}, (w, id, op, level, pos)))$  to  $BUF_1[j]$ 

```

3. 给 BUF_1 补充 dummy 元素, 让每个 bin/level 都被填满到最大容量。

- 1C/2C:

- 假设某个 bin 的容量是 b_i , 那么就往 BUF_1 里追加 b_i 个 dummy, 格式是 $(\perp, \perp, \perp, bin, 0)$ 。
- 对每个 bin 都执行一遍。
- 最终一共加了 $3 \cdot 2^i$ 个 dummy。

- NlogN:

- 在 level k , 有 2^{i-k} 个列表, 每个列表长度是 2^k 。
- 对于每个位置 pos (0 到 $2^{i-k} - 1$), 追加 2^k 个 dummy, 格式是 $(\perp, \perp, \perp, k, pos)$ 。
- 这个过程对所有 level $k \in \{0, \dots, i\}$ 都要执行。
- 最终一共加了 $2^i \cdot \log 2^i$ 个 dummy。

```

/** Phase 3—Add dummies */
14: for level = 0... $m_i - 1$  do
15:   for every  $pos \in \{0 \dots c_{level}\}$  call Client appends  $(\text{RND.Enc}(k_{rnd}, (\perp, \perp, \perp, level, pos)))$  to  $BUF_1$   $b_{level}$  times

```

4. 处理 BUF_1 , 用于生成 $NEW_i.IND$

把条目按 $(level, pos)$ 整理好, 用一次线性扫描打上标签替换原先的 $(level, pos)$, 去掉多余的 dummy, 得到新的倒排索引部分。

- 处理 BUF_2 , 用于生成 $NEW_i.DICT$

对 BUF_2 按随机密钥 p 升序做 oblivious sort。Dummy 条目 ($p = \perp$) 会自然排在最后。

接下来选取前 2^i 个条目: 因为第 i 层最多有 2^i 个不同关键字, 所以只取前 2^i 个非 dummy 条目来生成 $NEW_i.DICT$ 。

```

/** Phase 4—Final placement */
16: Perform oblivious sort on  $BUF_1$  w.r.t. the integer values of  $(level, pos)$  in ascending order
17: Linearly scan  $BUF_1$ , and tag first  $b_{level}$  entries for each  $level \in \{0, \dots, \Sigma m_i - 1\}$  and  $pos \in \{0, \dots, c_{level}\}$  with 1, and with 0 the rest of them (the existing  $(level, pos)$  tags are replaced with 0/1 tags, the entries now look like  $(w, id, op, x)$ , where  $x \in \{0, 1\}$ )
18: Perform order preserving oblivious compaction on  $BUF_1$ ; keep first  $N'$  entries; discard the 0/1 tags
19: Perform oblivious sort on  $BUF_2$  w.r.t. the random keys in ascending order; keep first  $2^i$  entries; discard random keys
20: Server moves  $BUF_1$  to  $NEW_i.IND$  ▷ elements in each bin/pos is randomly shuffled
21: for each  $s \in BUF_2$  do
22:   Client decrypts  $(w, cnt_w) \leftarrow \text{RND.Dec}(k_{rnd}, s)$ 
23:   Client generates  $(key, value) \leftarrow \text{PiBAS.Map}(P[i][3], k_{rnd}, w, cnt_w, 1)$  ▷ parameter 1 is used by the PRF  $F$ 
24:   Client writes  $NEW_i.DICT[key] \leftarrow value$ 
25: return  $NEW_i$ 

```


L-SDd[·] 方案的效率分析

简单来说，因为用到了 SD_d 方案，在访问时要查询 $3 \log N$ 个索引和字典，因此其 IO 效率要乘上一个 $\log N$ 因子

在 1C 方案下，不需要像 2C/NlogN 那样严格对 keyword-lists 做 padding（补齐到 2 的幂次），也不需要先存大列表再存小列表。

对 NlogN 来说，存储时“列表顺序并不重要”，因此第二次排序（line 6）可以省掉。

2C 的优化是 把 bin-usage map 转移到服务器端的 oblivious map，从而避免客户端存储过大。

SD_d ，它可以将任何隐藏结果的静态 SE 方案转换为前向/后向私有 DSE 方案。 SD_d 的本质是在 $\log N$ 个索引（从第 0 个到第 $(\log N - 1)$ 个）中存储 N 次更新的结果。第 i 个索引（ EDB_i ）的大小为 2^i 。每次更新时，客户端都会初始化静态 SE 方案（使用 SE 的设置），以生成并上传大小为 1 的加密索引（ EDB_0 ）。当两个相同大小的索引出现时（在服务器中），客户端下载、解密并将其组合成一个双倍大小的索引，从而分摊更新成本。在每个索引中独立执行搜索。

Scheme	Storage	Search			Update Cost	FP / BP	HDD / SSD /
		Locality	Read Efficiency	Page Efficiency			
LayeredSSE [55]	$O(N)$	$O(n_w/p + \log N)$	$O(\log \log N)$	$O(\log \log N)$	$O(n_w)$	✗ / ✗	SSD
Local[LayeredSSE] [55]*	$O(N)$	$O(1)$	$\tilde{O}(\log \log N)$	$\tilde{O}(\log \log N)$	$O(n_w)$	✗ / ✗	HDD
$SD_d[1C]$ [6]	$O(N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)(am.)$	✓ / II	HDD
$SD_d[2C]$ [6]*	$O(N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log^2 N)(am.)$	✓ / II	HDD
$SD_d[N \log N]$ [6]	$O(N \log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log^2 N)(am.)$	✓ / II	HDD/SSD
$SD_d[sN]$ [31]	$O(sN)$	$O(N^{\frac{1}{s}} + \log N)$	$O(\log N)$	$O(N^{\frac{1}{s}}/p + \log N)$	$O(s \log N)(am.)$	✓ / II	HDD/SSD
$SD_d[Tethys/Pluto]$ [10]	$O(N)$	$O(n_w/p + \log N)$	$O(p)$	$O(\log N)$	$O(N \log N)(am.)$	✓ / II	SSD
L- $SD_d[1C]$ [6]	$O(N)$	$O(\log N)$	$\tilde{O}(\log N)$	$\tilde{O}(\log N)$	$O(\log^2 N)$	✓ / II	HDD
L- $SD_d[2C]$ [6]*	$O(N)$	$O(\log N)$	$\tilde{O}(\log N)$	$O(\log N)$	$O(\log^2 N)$	✓ / II	HDD
L- $SD_d[N \log N]$ [6]	$O(N \log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log^3 N)$	✓ / II	HDD/SSD
L- $SD_d[sN]$ [31]	$O(sN)$	$O(N^{\frac{1}{s}} + \log N)$	$O(\log N)$	$O(N^{\frac{1}{s}}/p + \log N)$	$O(s \log^2 N)$	✓ / II	HDD/SSD

Figure 1: Comparison of different *locality-aware* and *page-efficient* DSE schemes. For any function $f(N)$ the notation $\tilde{O}(f(N))$ denote $O(f(N)(\log f(N))^x)$ for some constant x ; n_w denotes the number of updates for a keyword w , and p denotes memory page size and *am.* stands for amortized complexity. **FP** and **BP** stands for forward privacy and backward privacy respectively; all the schemes except the schemes from [55] satisfy **FP** and **BP-II**. The sources of the schemes that $SD_d[\cdot]$ and L- $SD_d[\cdot]$ are instantiated with are cited inside the brackets. The schemes marked with * restrict keyword-lists' size to be at most $N^{1-1/\log \log N}$.